CRICKET CLASH — DOC 5 OF 5 (FINAL)

PVP SYNC FIXES, 3D CHARACTER PROMPTS, SOUND ASSETS, CINEMATICS, PERFORMANCE

The Last Piece — Everything Not Covered in Docs 1-4

Repo: https://github.com/pyushmatania/handcricketgame-8ec46dab

---

CHAPTER 1: PVP SYNC FIXES — EVERY KNOWN BUG

1.1 BUG: TOSS GIVES BOTH PLAYERS THE SAME OPTION

Problem: Both PvP players see “even” or both see “odd” — no actual head-to-head coin call.

Fix:

```
// TossManager.ts

async function handleToss(matchId: string, userId: string, opponentId: string) {
  // 1. BOTH players see the coin — but only ONE calls it
  // The "host" (player who created the match) CALLS the toss
  // The other player WATCHES

  if (isHost) {
    // Show HEADS/TAILS buttons
    const call = await waitForUserTap(); // 'heads' or 'tails'

    // Send call to server
    await syncEngine.channel.send({
      type: 'broadcast', event: 'toss_call',
      payload: { userId, call }
    });
  } else {
    // NON-HOST: Show "Opponent is calling the toss..." with spinner
    // Wait for host's call
    const hostCall = await waitForBroadcast('toss_call');
    // Show: "Opponent called HEADS"
  }

  // 2. SYNC POINT — both players ready
  await syncEngine.waitForSync('toss_flip');

  // 3. Server flips the coin (RANDOM, not dependent on either player)
  let coinResult: 'heads' | 'tails';
  if (isHost) {
    coinResult = Math.random() < 0.5 ? 'heads' : 'tails';
    await syncEngine.channel.send({
      type: 'broadcast', event: 'toss_result',
      payload: { result: coinResult }
```

```
    });
  } else {
    const resultPayload = await waitForBroadcast('toss_result');
    coinResult = resultPayload.result;
  }

  // 4. Both players see the SAME coin animation and result
  await playCoinFlipAnimation(coinResult);

  // 5. Winner chooses BAT/BOWL
  const tossWinner = (hostCall === coinResult) ? hostId : opponentId;

  if (tossWinner === userId) {
    // Show BAT/BOWL buttons
    const decision = await waitForUserTap(); // 'bat' or 'bowl'
    await syncEngine.channel.send({
      type: 'broadcast', event: 'toss_decision',
      payload: { decision, winnerId: userId }
    });
  } else {
    // Show "Opponent won the toss. Waiting for their decision..."
    const decisionPayload = await waitForBroadcast('toss_decision');
    // Show: "Opponent chose to BAT FIRST"
  }

  // 6. SYNC POINT — both ready to start
  await syncEngine.waitForSync('toss_complete');
}
```

Key principle: Only ONE player makes a choice at each step. The other WATCHES and WAITS. This prevents the “both see the same thing” bug.

1.2 BUG: MATCH LOADS FASTER FOR ONE PLAYER

Problem: One player enters the match screen before the other. They select their number, but the opponent hasn’t loaded yet.

Fix: Mandatory sync point before EVERY interactive moment:

```
// MatchController.ts

async function startMatch() {
  // 1. Both players load match assets
  await loadMatchAssets(theme, weather, players);

  // 2. SYNC: Both must be loaded before proceeding
  await syncEngine.waitForSync('match_loaded', 30000); // 30s timeout

  // 3. Show VS screen (both see it at the same time)
  await showVSScreen(); // 3 seconds
  await syncEngine.waitForSync('vs_done');

  // 4. Toss (fully synced — see 1.1)
  await handleToss();
  await syncEngine.waitForSync('toss_complete');

  // 5. Team walkout animation (both see it simultaneously)
  await showTeamWalkout(); // 3 seconds
  await syncEngine.waitForSync('walkout_done');

  // 6. FIRST BALL — both players' hand selectors activate at the SAME moment
  await syncEngine.waitForSync('first_ball_ready');
  enableHandSelector();
}
```

1.3 BUG: EVENTS HAPPEN TOO FAST / OUT OF ORDER

Problem: Ball result, over break card, innings change, and other events stack up and happen simultaneously, confusing the player.

Fix: The Event Engine (Doc 2) with BLOCKING events and MANDATORY durations solves this completely. But here's the additional rule:

```
// RULE: After any blocking event completes, insert a BREATHING PAUSE of 300ms
// before the next event starts. This prevents the "rapid-fire" feeling.

private async processNext(): Promise<void> {
  // ... (existing processing logic from Doc 2)

  if (this.currentEvent.blocking && this.currentEvent.duration > 0) {
    await this.sleep(this.currentEvent.duration);
    // ← NEW: Breathing pause between blocking events
    await this.sleep(300);
  }

  this.processNext();
}
```

1.4 BUG: MATCH DOESN'T START SOMETIMES

Problem: In PvP, occasionally the match just hangs and never starts.

Fix: Implement a **connection health check** with automatic recovery:

```
class ConnectionMonitor {
  private heartbeatInterval: NodeJS.Timeout | null = null;
  private lastHeartbeat: number = Date.now();
  private maxHeartbeatAge: number = 10000; // 10 seconds

  start(channel: RealtimeChannel, userId: string) {
    // Send heartbeat every 3 seconds
    this.heartbeatInterval = setInterval(() => {
      channel.send({
        type: 'broadcast', event: 'heartbeat',
        payload: { userId, timestamp: Date.now() }
      });
    }, 3000);

    // Listen for opponent heartbeats
    channel.on('broadcast', { event: 'heartbeat' }, ({ payload }) => {
      if (payload.userId !== userId) {
        this.lastHeartbeat = Date.now();
      }
    });

    // Check for stale connection every 5 seconds
    setInterval(() => {
      if (Date.now() - this.lastHeartbeat > this.maxHeartbeatAge) {
        this.handleDisconnect();
      }
    }, 5000);
  }

  private handleDisconnect() {
    // Show toast: "Connection lost. Attempting to reconnect..."
    // Try to re-subscribe to channel
    // If fails after 3 attempts (15 seconds):
    //   "Opponent disconnected. Match will continue vs AI."
    //   Switch to AI mode for remaining match
```

```
  }

  stop() {
    if (this.heartbeatInterval) clearInterval(this.heartbeatInterval);
  }
}
```

1.5 BUG: AFTER EACH BALL, THINGS RUSH FORWARD

Problem: The ball result shows for a split second, then immediately the next ball starts. No time to process what happened.

Fix: The timing table in Doc 2 (Event Engine) enforces minimum durations. But additionally, after EVERY ball result animation completes, the hand selector buttons should NOT re-enable instantly:

```
// After ball result animation completes:
async function prepareNextBall() {
  // 1. Wait for all animations to finish (enforced by Event Engine)

  // 2. Brief pause (500ms) — let the player breathe
  await sleep(500);

  // 3. Check game state (is over complete? innings over? match over?)
  if (isOverComplete()) {
    // Show Over Break card (blocks until dismissed)
    await showOverBreakCard();
  }

  if (isInningsComplete()) {
    // Show Innings Break card (blocks until both players ready)
    await showInningsBreakCard();
  }

  if (isMatchComplete()) {
    // Transition to result screen
    return;
  }

  // 4. Another brief pause (300ms) before re-enabling buttons
  await sleep(300);

  // 5. NOW re-enable the hand selector
  // 6. Start bowler run-up animation
  eventEngine.emit('BALL_BOWLER_RUNUP', {});
}
```

1.6 BUG: NOTIFICATIONS/INVITES NOT WORKING PROPERLY

Fix: Use Supabase Database Webhooks + Push API:

```
// Listen for new tournament invites in real-time
supabase
  .channel('user-notifications')
  .on('postgres_changes', {
    event: 'INSERT',
    schema: 'public',
    table: 'notifications',
    filter: `user_id=eq.${currentUserId}`
  }, (payload) => {
    showNotificationToast(payload.new);
    // Play notification sound
    soundEngine.playEffect('ui_notification');
    // Vibrate
```

```
    soundEngine.vibrate('light');
  })
  .subscribe();
```

Add a `notifications` table:

```
CREATE TABLE notifications (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES user_profiles(id),
  type TEXT NOT NULL, -- 'tournament_invite', 'war_started', 'chest_ready', etc.
  title TEXT NOT NULL,
  body TEXT NOT NULL,
  data JSONB, -- Additional context (tournament_id, match_id, etc.)
  read BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

CHAPTER 2: 3D CHARACTER PROMPTS — TOP 50 PLAYERS

These are ready-to-paste prompts for Midjourney, Leonardo AI, Flux, or DALL-E. Each prompt generates a Supercell-style 3D character. Generate at 512×512 minimum with transparent background.

2.1 MYTHIC TIER (5 Players)

VIRAT KOHLI — "King Kohli" (RCB)

```
3D render of Indian cricketer Virat Kohli as a Supercell Clash of Clans style character, chunky exaggerated proportior
```

MS DHONI — "Thala" (CSK)

```
3D render of Indian cricketer MS Dhoni as a Supercell Clash of Clans style character, chunky proportions big head, wea
```

ROHIT SHARMA — "Hitman" (MI)

```
3D render of Indian cricketer Rohit Sharma as a Supercell style character, chunky proportions, wearing Mumbai Indians
```

JASPRIT BUMRAH — "Boom Boom" (MI)

```
3D render of Indian cricketer Jasprit Bumrah as a Supercell style character, UNIQUE body shape emphasizing his unusual
```

ANDRE RUSSELL — "Dread Russell" (KKR)

```
3D render of Jamaican cricketer Andre Russell as a Supercell style character, EXTREMELY muscular build with oversized
```

2.2 LEGENDARY TIER (15 Players)

RASHID KHAN (GT)

```
3D render of Afghan cricketer Rashid Khan as a Supercell style character, youthful boyish face with intense focused ey
```

RISHABH PANT (LSG)

3D render of Indian cricketer Rishabh Pant as a Supercell style character, energetic youthful round face with big chee

SURYAKUMAR YADAV (MI)

3D render of Indian cricketer Suryakumar Yadav as a Supercell style character, athletic confident build, wearing Mumba

HEINRICH KLAASEN (SRH)

3D render of South African cricketer Heinrich Klaasen as a Supercell style character, stocky powerful build with thick

PAT CUMMINS (SRH)

3D render of Australian cricketer Pat Cummins as a Supercell style character, tall athletic build, wearing Sunrisers H

SHUBMAN GILL (GT)

3D render of Indian cricketer Shubman Gill as a Supercell style character, handsome youthful face with stylish hair vi

JOS BUTTLER (GT)

3D render of English cricketer Jos Buttler as a Supercell style character, athletic build, wearing Gujarat Titans jers

SANJU SAMSON (RR)

3D render of Indian cricketer Sanju Samson as a Supercell style character, athletic build with broad shoulders, wearir

YASHASVI JAISWAL (RR)

3D render of Indian cricketer Yashasvi Jaiswal as a Supercell style character, young lean athletic build, wearing Raja

TRAVIS HEAD (SRH)

3D render of Australian cricketer Travis Head as a Supercell style character, athletic build, wearing Sunrisers Hydera

KL RAHUL (DC)

3D render of Indian cricketer KL Rahul as a Supercell style character, elegant slim build, wearing Delhi Capitals blue

GLENN MAXWELL (PBKS)

3D render of Australian cricketer Glenn Maxwell as a Supercell style character, medium athletic build, wearing Punjab

JOFRA ARCHER (RR)

3D render of Barbadian-English cricketer Jofra Archer as a Supercell style character, athletic lean build, wearing Raj

MOHAMMED SHAMI (SRH)

3D render of Indian cricketer Mohammed Shami as a Supercell style character, strong build with broad shoulders, wearir

RAVINDRA JADEJA (CSK)

3D render of Indian cricketer Ravindra Jadeja as a Supercell style character, athletic fit build, wearing Chennai Supe

2.3 EPIC TIER (15 Key Players — Shortened Prompts)

For each of these, use the same base structure: `3D render of [name] as a Supercell Clash of Clans style character, chunky proportions big head, wearing [team] jersey, [signature pose], [expression], [skin tone], transparent background, full body, 512x512`

Player	Team	Signature Pose	Key Visual Detail
Tilak Varma	MI Blue	Left-hand drive	Young face, left-hand bat
Rinku Singh	KKR Purple	Arms raised celebrating	Emotional expression, remembering his 5 sixes moment
Phil Salt	RCB Red	Explosive pull shot	Helmet, aggressive stance
Tim David	RCB Red	Huge slog over cow corner	Tall, long reach, bat extended high
Hardik Pandya	MI Blue (captain)	Bowling action mid-delivery	Captain armband, athletic, dual role
Shreyas Iyer	PBKS Red (captain)	Cover drive on front foot	Composed captain, neat appearance
Liam Livingstone	RCB Red	Switch hit (both sides)	Ambidextrous energy, wild shot
Axar Patel	DC Blue (captain)	Left-arm spin delivery	Tall, left-arm orthodox action
Kuldeep Yadav	DC Blue	Chinaman bowling action	Left-arm wrist spin, deceptive
Jake Fraser-McGurk	DC Blue	Brutal pull shot	Young aggressive face, massive hit
Abhishek Sharma	SRH Orange	Left-hand big hit	Young, opening batsman energy
Ishan Kishan	SRH Orange	Quick-fire batting stance	Small but dynamic, behind stumps
Mitchell Starc	DC Blue	Yorker delivery stride	Tall, left-arm fast, 155kph aura
Trent Boult	MI Blue	Swinging delivery	Left-arm fast, ball curving in air
Varun Chakravarthy	KKR Purple	Mystery spin grip shown	Glasses sometimes, mysterious aura

2.4 THUMBNAIL GENERATION

For each character, also generate a **circular thumbnail** version:

```
Same character as above but PORTRAIT ONLY — head and upper chest, circular crop, facing slightly left, [team jersey] c
```

2.5 SPECIAL POSES (Additional Per Character)

For the top 10 players, generate additional pose variants:

Pose	When Used	Description
Batting stance	Match screen (on strike)	Ready position, bat raised, eyes on bowler
Celebration	After scoring 50/100, after winning	Unique per player (Kohli fist pump, Dhoni bat twirl, etc.)
Walking back dejected	After getting out	Head down, bat dragging, slow walk
Bowling action	Match screen (bowling)	Mid-delivery stride
Fielding dive	Caught animation	Horizontal dive, one-handed
Team walkout	Pre-match lineup	Walking confidently onto the field

CHAPTER 3: COMPLETE SOUND ASSET GUIDE

3.1 HOW TO SOURCE/GENERATE ALL 87 SOUND FILES

Option A: AI Sound Generation (Recommended)

Use **ElevenLabs Sound Effects**, **Soundraw**, or **Mubert** to generate custom sounds:

Sound Category	Generation Prompt	Post-Processing
Bat hits (soft)	"Cricket bat gently tapping a leather ball, soft contact, indoor net practice"	Normalize volume, trim to 0.3s
Bat hits (hard)	"Powerful cricket bat hitting ball, loud crack, outdoor stadium echo"	Normalize, add slight reverb, trim to 0.5s
Bat hits (massive)	"Explosive cricket bat smashing ball for a six, thunderous crack, crowd gasps"	Boost bass, add shockwave sub-bass, trim to 0.6s
Stumps hit	"Cricket ball smashing into wooden stumps, timber flying, sharp rattle and crash"	Keep raw and impactful, trim to 0.8s
Ball into gloves	"Cricket ball landing in wicketkeeper gloves, leather thud, satisfying catch sound"	Normalize, trim to 0.3s
Crowd cheer (mild)	"Small crowd applauding at a cricket match, polite clapping"	Loop-safe, 2 seconds
Crowd eruption	"Massive cricket stadium crowd exploding with joy after a six, 50000 people roaring"	Keep long (3s), normalize peaks
Crowd groan	"Cricket crowd groaning in disappointment, collective sigh"	1.5 seconds
Appeal	"Cricket fielders appealing for a wicket, multiple voices shouting HOWZAT"	Trim to 1.5s, multiple variants at different intensities
Coin flip	"Metal coin being flipped in the air, spinning metallic ring"	Trim to 1s
Fireworks	"Single firework bursting in the sky, pop and sparkle"	Trim to 1s

Option B: Free Sound Libraries

Source	URL	Best For
Freesound.org	freesound.org	Crowd sounds, ambient, nature
Pixabay Audio	pixabay.com/sound-effects	UI sounds, transitions
Zapsplat	zapsplat.com	Sports sounds, impacts
BBC Sound Effects	sound-effects.bbcrewind.co.uk	Cricket-specific (real match recordings)
YouTube Audio Library	youtube.com/audiolibrary	Background music

Option C: Record Custom

For the most authentic cricket sounds:

- Record actual bat-ball contacts at a cricket net session
- Record crowd sounds at a local cricket match
- Record coin flips, button clicks, etc. with a smartphone
- Post-process in Audacity (normalize, trim, export as MP3 at 128kbps)

3.2 SOUND FILE SPECIFICATIONS

Spec	Value
Format	MP3 (128kbps) or WebM/OGG for web
Sample Rate	44100 Hz
Channels	Mono (effects) or Stereo (ambient/music)
Max Duration	Effects: 0.3-3s. Ambient loops: 10-30s. Music: 60-120s
Total Files	87 unique files
Total Size	~15-25 MB (all sounds combined)
Naming	`category_variant_number.mp3` (e.g., `bat_hit_hard_3.mp3`)

3.3 COMMENTARY VOICE GENERATION

For pre-recorded commentary lines (higher quality than Web Speech API):

Using ElevenLabs

1. Create 2 voice profiles:
   - **Main Commentator:** Male, Indian English accent, age 35-45, energetic, clear diction
   - **Color Commentator:** Male, Indian English accent, age 50-60, calmer, analytical
2. Generate all 350 English lines as individual MP3 files
3. Organize in folders: `/sounds/commentary/english/four/` , `/sounds/commentary/english/six/` , etc.

Using Web Speech API (Fallback — Already Coded in Doc 2)

- Uses device's built-in TTS
- Set `lang: 'en-IN'` for Indian English accent
- Quality varies by device — some Android devices have excellent Hindi TTS

Hybrid Approach (Recommended)

- **Pre-record the top 50 most-used lines** (all SIX lines, all WICKET lines, match start/end) as high-quality MP3
- **Use TTS for the remaining 300 lines** (dots, singles, over summaries — less critical quality)
- This gives the best balance of quality vs. file size

CHAPTER 4: PRE/POST MATCH CINEMATICS — EXPANDED

4.1 DRESSING ROOM SCENE (Pre-Match Lobby Detail)

When waiting for an opponent in the lobby, your character is in a **dressing room**. This environment should feel like a real cricket team's changing room:

Elements in the Dressing Room:

- **Lockers:** 3-4 metal lockers in the background, one open showing your team jersey hanging inside
- **Bench:** A wooden bench with cricket gear scattered on it — pads, gloves, a helmet, a water bottle
- **Cricket bat:** Leaning against the wall next to the locker
- **Team banner/flag:** Your team's logo/flag on the wall
- **Shoes:** Cricket spikes on the floor near the bench
- **Whiteboard:** A tactics whiteboard in the corner with pitch diagram drawn on it (barely visible, adds authenticity)

Character Idle Animations in Dressing Room:

```
Animation 1 (3s loop): Strapping on batting pads — reaches down, wraps pad around shin, pulls strap
Animation 2 (2s loop): Putting on batting gloves — pulls glove onto hand, flexes fingers
Animation 3 (4s loop): Shadow batting — takes a practice swing in the confined space, nearly hits a locker
Animation 4 (3s loop): Stretching — reaches arms overhead, touches toes, rotates shoulders
Animation 5 (5s loop, rare): Looks at phone, puts it down, looks nervous — building anticipation
```

These cycle randomly. When opponent is found, the character STOPS what they're doing, looks up toward camera, nods, and the scene transitions to the VS screen.

3D Asset Prompt — Dressing Room Background:

```
3D render of a cricket team dressing room interior, Supercell art style, a row of metal lockers against the back wall
```

4.2 STADIUM JUMBOTRON MOMENT

Before the VS screen, a 2-second shot of the ground's giant LED screen/jumbotron:

```
Camera Position: Behind the bowler's arm, looking toward the far end of the ground
What's Visible: The pitch in the foreground (out of focus), the far stand with the GIANT LED screen
On the LED Screen: The VS graphic is displayed (both team logos, diagonal split, "VS")
The camera ZOOMS INTO the LED screen over 2 seconds until the VS graphic fills the entire phone screen
This creates a seamless transition: real stadium view → zoom into jumbotron → VS screen IS the jumbotron content
```

Sound during zoom: Stadium PA system playing dramatic intro music, crowd building.

4.3 POST-MATCH PRESS CONFERENCE (Theme-Adaptive)

After the Match Result screen, an optional "POST-MATCH" section plays:

Professional Theme (Stadium/IPL/International)

```
Scene: Press conference room with microphones, camera flashes, sponsor board behind
Characters: Winning captain at the mic, journalist characters in seats
Captain speaks (commentary engine): "It was a great team effort. We executed our plans well."
Player of the Match stands next to captain with trophy
```

```
Camera flashes pop (white circles flashing randomly)
Duration: 5 seconds, tap to skip
```

Gully Theme

```
Scene: Same gully lane, everyone standing around arguing
Characters: Winners celebrating, losers making excuses
Commentary (Hinglish): "Bhai ne kya batting ki! Tera bowling toh bakwas tha!"
Loser character: Arms folded, looking away annoyed
Winner character: Dancing, mocking, pointing and laughing
Duration: 4 seconds, tap to skip
```

School Ground Theme

```
Scene: School corridor, principal's office door visible
Characters: Teacher character hands out a chocolate/star to the Player of the Match
Commentary: "Aaj ka star student — sabse accha khela! Gold star!"
Other kids clapping and cheering
Duration: 4 seconds, tap to skip
```

Beach Theme

```
Scene: Sunset beach, players sitting on sand with coconut drinks
Characters: Relaxed, high-fiving, watching sunset
Commentary: "Mast game tha yaar. Chalo ab chill karte hai."
Duration: 3 seconds, tap to skip
```

CHAPTER 5: PERFORMANCE OPTIMIZATION

5.1 ASSET LOADING STRATEGY

```
// AssetManager.ts — Prioritized asset loading

const LOAD_PRIORITY = {
  CRITICAL: 1,  // Must load before anything shows (fonts, core CSS, home screen bg)
  HIGH: 2,      // Load during splash screen (UI icons, button textures, home island)
  MEDIUM: 3,    // Load on navigation (match assets loaded during VS screen)
  LOW: 4,       // Lazy load (character full arts, shop items, leaderboard)
  DEFERRED: 5,  // Load on demand (AR assets, replay data, achievement badges)
};

class AssetManager {
  async preloadCritical() {
    await Promise.all([
      loadFont('Bungee'),
      loadFont('Lilita One'),
      loadFont('Nunito'),
      loadFont('Rubik'),
      loadImage('/assets/ui/chrome-bracket.png'),
      loadImage(`/assets/islands/arena_${currentArena}.png`),
    ]);
```

```
  }

  async preloadMatch(theme: string, weather: string) {
    // Called during VS screen (3 seconds to load)
    await Promise.all([
      loadImage(`/assets/pitch/${theme}_pitch.png`),
      loadImage(`/assets/pitch/${theme}_bg.png`),
      loadAudio('bat_hit_hard_1.mp3'),
      loadAudio('bat_hit_massive_1.mp3'),
      loadAudio('crowd_eruption_1.mp3'),
      loadAudio('stumps_hit_1.mp3'),
      loadAudio(`ambient_crowd_quiet_loop.mp3`),
    ]);
  }
}
```

5.2 IMAGE OPTIMIZATION

Asset Type	Format	Max Size	Compression
UI icons/brackets	WebP	10KB	Lossless
Character thumbnails	WebP	30KB	Quality 85%
Character full art	WebP	100KB	Quality 80%
Pitch view backgrounds	WebP	200KB	Quality 75%
Island renders	WebP	150KB	Quality 80%
Establishing shot panoramas	JPEG	300KB	Quality 70%

Total estimated asset size: ~50-80 MB (all images + sounds)

CDN Strategy: Host on Supabase Storage with global CDN. First-time load downloads assets progressively. Subsequent loads use browser cache.

5.3 ANIMATION PERFORMANCE

```
// Rules for smooth 60fps animations:

// 1. Only animate transform and opacity (never width, height, top, left)
// 2. Use will-change on elements that animate frequently
// 3. Use requestAnimationFrame for custom animations
// 4. Debounce scroll events (50ms)
// 5. Use React.memo on static components (scoreboard updates only on score change)
// 6. Lazy-load heavy components (scorecard charts, AR mode)

// 7. Particle system limits:
const PARTICLE_LIMITS = {
  rain: { maxCount: 150, fps: 30 },      // Reduce for low-end devices
  dust: { maxCount: 100, fps: 30 },
  confetti: { maxCount: 200, fps: 60 },  // Short burst, ok to be higher
  fireworks: { maxCount: 80, fps: 60 },
};

// 8. Detect low-end devices:
function isLowEndDevice(): boolean {
  const memory = (navigator as any).deviceMemory;
  const cores = navigator.hardwareConcurrency;
  return (memory && memory <= 4) || (cores && cores <= 4);
}
```

```
// If low-end: reduce particle counts by 50%, disable parallax, reduce shadow complexity
```

5.4 MEMORY MANAGEMENT

```
// Clean up heavy resources when navigating away from screens:

useEffect(() => {
  return () => {
    // Cleanup on unmount
    soundEngine.stopAllEffects();
    weatherEngine.clearParticles();
    // Release large image references
    pitchViewRef.current = null;
  };
}, []);

// For match screen: preload on enter, cleanup on exit
// For collection screen: virtualize the card grid (only render visible cards)
// For leaderboard: paginate (load 20 at a time, load more on scroll)
```

5.5 BUNDLE SIZE OPTIMIZATION

```
// vite.config.ts
export default defineConfig({
  build: {
    rollupOptions: {
      output: {
        manualChunks: {
          'engines': ['./src/engines/EventEngine', './src/engines/SyncEngine', ...],
          'match': ['./src/pages/Match', './src/components/match/*'],
          'tournament': ['./src/pages/Tournament', './src/components/tournament/*'],
          'ar-mode': ['three', '@mediapipe/hands'], // Separate chunk, lazy loaded
        }
      }
    },
    chunkSizeWarningLimit: 500, // KB
  }
});
```

Target bundle sizes:

- Initial load (home screen): < 300KB JS + 200KB CSS
- Match screen chunk: < 400KB JS
- Tournament chunk: < 200KB JS
- AR mode chunk: < 800KB JS (only loaded if user enables AR)

CHAPTER 6: ACCESSIBILITY & SETTINGS

6.1 SETTINGS SCREEN

```
┌── SETTINGS ──────────────────────────────────────┐
│                                                  │
│  🔊 Sound Effects        [▓▓▓▓▓▓▓▓▒▒] 80%   [ON/OFF] │
│  🎵 Music                [▓▓▓▒▒▒▒▒▒▒] 40%   [ON/OFF] │
│  🎙 Commentary           [▓▓▓▓▓▓▓▓▓▓] 100%  [ON/OFF] │
│  📳 Vibration                               [ON/OFF] │
│                                                  │
│  🗣 Commentary Language                           │
│  ┌─────────┐ ┌─────────┐ ┌─────────┐              │
│  │ English │ │  Hindi  │ │ Hinglish │              │
│  │   ✅    │ │         │ │         │              │
│  └─────────┘ └─────────┘ └─────────┘              │
│                                                  │
│  🎨 Theme (Home Island)                           │
│  Current: Gully Cricket  [CHANGE →]               │
│                                                  │
│  🏏 Selected Team                                 │
│  Current: CSK  [CHANGE →]                         │
│                                                  │
│  👤 Account                                       │
│  Username: Akash  [EDIT]                          │
│  Link Account: [Google] [Apple]                   │
│                                                  │
│  🔄 Reduced Motion                    [ON/OFF]    │
│  (Disables screen shake, particles)               │
│                                                  │
│  📱 About                                         │
│  Version: 2.0.0                                   │
│  [Privacy Policy] [Terms] [Support]               │
│                                                  │
│  🗑 [Clear Cache]  [Delete Account]               │
│                                                  │
└──────────────────────────────────────────────────┘
```

Volume sliders: Horizontal sliders with team-colored fill. Drag to adjust. The value updates in real-time (can hear the effect while adjusting).

Language selector: Three buttons (radio style — only one active). Changing language immediately affects commentary in the next match.

Theme selector: Opens a carousel showing available themes (locked themes are greyed with "Unlock at Level X" overlay).

Team selector: Opens the team selection carousel (10 team logos, tapping one changes the entire UI theme — see Doc 1 dynamic theming).

Reduced Motion: When ON, disables: screen shake, particle effects, parallax, breathe animations. Keeps: slide transitions, fades, score updates (functional animations).

CHAPTER 7: HIDDEN FEATURES PAGE (Your Request)

7.1 CRICKET CLASH INFO SCROLL

A hidden page accessible by tapping the version number 5 times in Settings. This page auto-scrolls through all features of the app in a cinematic way:

```
// InfoScroll.tsx — Hidden feature showcase

const FEATURES = [
  { title: 'Game Engines', items: ['Event Engine', 'Sync Engine', 'Gameplay Engine', 'Sound Engine', 'Commentary Engir
  { title: 'Game Modes', items: ['T3 Blitz', 'T5 Quick', 'T10 Classic', 'T20 Full', 'Super Over'] },
  { title: 'Tournament Formats', items: ['IPL Format', 'World Cup', 'Ashes', 'Knockout Cup', 'Auction League', 'Cricke
  { title: 'Cricket Wars', items: ['Stadium Defense', 'War Map', 'Traps System', 'War Leagues (10 tiers)'] },
  { title: 'Players', items: ['200+ Real IPL Players', '10 IPL Teams', 'International Teams', '5 Rarity Tiers', '15+ S
  { title: 'Themes', items: ['Gully Cricket', 'School Ground', 'Rooftop', 'Beach', 'Village Maidan', 'Night Street', '
  { title: 'Commentary', items: ['English', 'Hindi', 'Hinglish', 'Duo Commentary', 'Theme-Adaptive Tone', '1100+ Lines
  { title: 'Sound', items: ['87 Unique Sound Files', 'Perspective Audio (different for each player)', '38 Sound Categc
  { title: 'Progression', items: ['7 Arenas', '30 Levels', '7 Chest Tiers', '60-Tier Battle Pass', '60+ Achievements',
  { title: 'Social', items: ['Clan System (Level 1-10)', 'Card Donations', 'Tournament Chat', 'Emotes', 'Spectator Moc
  { title: 'AR Mode', items: ['Camera-Based AR Cricket', 'Hand Gesture Detection', '3D Miniature Ground on Real Surfac
  { title: 'Visual', items: ['V1 Punchy Colorful Style', '3D Chrome Frame System', 'Dynamic Team Theming', '3D Floatir
];
```

Display: Each category fades in one at a time, with the title in Lilita One and items cascading below. Auto-scrolls with ambient music. A “EXPORT PDF” button at the bottom generates a designed PDF of all features using the browser’s print functionality styled with the game’s visual identity.

CHAPTER 8: FINAL IMPLEMENTATION CHECKLIST

PHASE 1: Foundation (Week 1-2)

- Set up project structure (Doc 2 file structure)
- Implement Event Engine + Sync Engine
- Apply global CSS system (Doc 1 tokens, components)
- Home screen with player bar + chest banners + 3D island + BATTLE button + chest slots + tab bar
- Player profile creation + Supabase auth

PHASE 2: Core Gameplay (Week 3-4)

- Mode select drawer
- Toss screen (fully synced for PvP)
- VS screen (full + lobby + mini fixture version)
- Match screen: scoreboard + pitch view (1 theme first) + hand selector
- Ball result animations (dot/runs/four/six/wicket)
- Over break card, wicket card, innings change card, boundary flashes
- Match result screen (win/loss)
- Scorecard screens (6 pages with charts)
- Gameplay Engine with stat-modified ball resolution

PHASE 3: Sound & Commentary (Week 5)

- Sound Engine with all 87 files loaded

- Perspective audio system (PvP)
- Commentary Engine with English pool (350 lines)
- Duo commentary implementation
- Hindi + Hinglish pools
- Lighting Engine
- Weather Engine with particle systems
- Crowd Engine with mood system

PHASE 4: Players & Collection (Week 6)

- Seed 200+ players into Supabase
- Player card component (all 5 rarities)
- Collection screen (grid + filters + detail view)
- Team builder (formation + carousel + presets)
- Player upgrade system

PHASE 5: Progression (Week 7)

- Chest system (4 slots, timers, opening animation)
- Arena system (trophies, promotion/demotion)
- XP / level system
- Daily quests
- Battle pass (60 tiers)
- Achievements (60+ badges)
- Shop screen
- Leaderboard

PHASE 6: Social & Tournaments (Week 8-9)

- Friends tournament creation + lobby + invites
- IPL Format tournament (round robin + playoffs)
- Points table + Orange/Purple Cap
- Knockout Cup format
- PvP matchmaking + ranked play
- Clan system (create, join, chat, donations)
- Tournament chat

PHASE 7: Advanced Features (Week 10-11)

- Cricket Wars (stadium defense + war map + attacks)
- War Leagues (monthly)
- Auction League format
- Cricket Royale format
- All 10 themes (pitch views + establishing shots)

- Pre-match cinematic (walkout + jumbotron)
- Post-match presentation

PHASE 8: Polish & AR (Week 12)

- Generate all 50 3D character assets
- Generate all 10 island assets
- Hand Cricket Trophy design + ceremony animation
- Trophy cabinet in profile
- Player skins + emotes
- AR mode (camera + 3D ground placement)
- Hand tracking for AR
- Seasonal themes (Diwali, Holi, IPL season)
- Spectator mode
- Replay theater
- Quick-chat in PvP
- Notifications system
- Performance optimization pass
- Bug testing across devices
- Production deployment

CHAPTER 9: SUMMARY — ALL 5 DOCUMENTS

Document	Content	Words
Doc 1	UI System — design tokens, 11 components with CSS, 16 screens pixel-level, transitions, micro-interactions, 22 asset prompts, responsive rules	~9,300
Doc 2	Engine Architecture — 8 engines with full TypeScript, 60+ event types, millisecond timing tables, ball sequence, 87 sound files, PvP sync protocol, gameplay math, EngineManager wiring	~11,600
Doc 3	Gameplay Rebuild — pre-match cinematic, 3D pitch view for 10 themes, ball animations (frame-by-frame for all result types), 1,100+ commentary lines (EN/HI/Hinglish), post-match presentation, AR mode with hand tracking, special events	~9,500
Doc 4	Social & Progression — 200+ real players, 6 tournament formats, Cricket Wars, clans, chests, upgrades, arenas, battle pass, achievements, skins, emotes, Hand Cricket Trophy, shop, leaderboard, 20+ Supabase tables	~5,900
Doc 5	Final — PvP bug fixes (6 bugs with code fixes), 3D prompts for 35 star players, sound sourcing guide, dressing room/jumbotron cinematics, performance optimization, settings, hidden features, 12-week implementation checklist	~5,700
TOTAL	**Complete game specification**	**~42,000**

These 5 documents contain everything needed to build Cricket Clash from scratch. Feed them to your builder in order (Doc 1 → 2 → 3 → 4 → 5). The UI style is V1 punchy/colorful with 3D chrome frames. Every screen, every engine, every player, every animation, every sound, every tournament format, and every progression mechanic is specified.

This is the final document. The complete Cricket Clash game bible is now 5 documents totaling approximately 42,000 words. Every pixel, every millisecond, every sound variant, every commentary line, every player stat, every tournament rule, every sync point, and every animation frame is documented. Go build the greatest cricket game ever made.